



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 3 · Цвет, гистограммы и точечные операции

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиплярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L3: Цвет, гистограммы и точечные операции

RECAP

Цифровой кадр не является точной копией реальности, а представляет собой результат математических допущений и физических ограничений:

- **Иллюзия цвета:** Камера фиксирует не спектр, а лишь три его интеграла (RGB). При этом **2/3 цветowych данных «дорисовываются» алгоритмом демозаики**, а гамма и баланс белого — это лишь догадки ISP-процессора под особенности человеческого глаза.
- **Природа шума:** Шум сильнее всего проявляется в тенях ($1/\sqrt{N}$). Любое высветление темных участков неизбежно **усиливает и растягивает шум**.
- **Геометрия и артефакты:** базовые расчеты: связь угла обзора и фокусного расстояния в пикселях, подавление шума путем усреднения кадров ($1/\sqrt{N}$) и природа геометрических искажений из-за строчного переноса кадра (Rolling Shutter).

Разбор самопроверки L2

- Камера 90° и 1280 px даёт $f_x = 640$ px, при этом дверь 2 м на дистанции 8 м отображается в 160 px
- Стек 16 кадров режет шум в 4x ($\sigma/4$), а 64 кадра – в 8x ($\sigma/8$)
- В кадре 8×8 RGGB: измерено 32 зеленых пикселя (G), демозаик выдумывает $32 G + 48 R + 48 B$
- Доски забора на Rolling Shutter наклонятся: строки сняты в разное время

LEARNING OBJECTIVES

- **Выбор цветового пространства:** Использовать **HSV** для создания масок по цвету, **Lab** для расчета цветовых расстояний и **YCrCb** для выделения яркостной компоненты.
- **Анализ и коррекция кадра:** Уметь оценивать экспозицию и контраст по гистограмме, устранять клиппинг методами **LUT**, эквализации или **CLAHE**, а также находить оптимальный порог бинаризации с помощью метода **Otsu**.
- **Построение конвейера обработки:** Собирать цепочку алгоритмов: выделение диапазона цвета (`inRange`) → очистка маски от шумов (морфология) → поиск объектов (`connectedComponentsWithStats`) → выделение их рамкой (`bbox`).
- **Анализ ошибок:** Понимать слабые места классических алгоритмов (смена освещения, слипшиеся объекты) и уметь их «лечить».

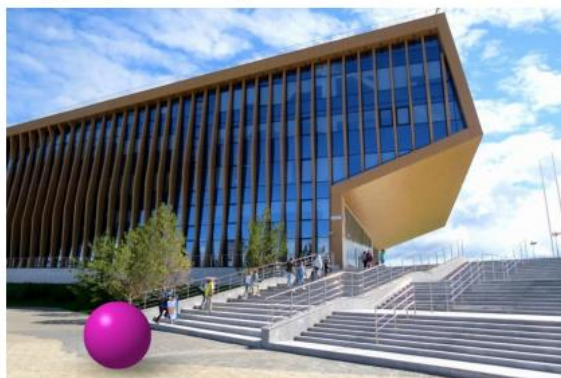
Задача: найти мяч при любом свете

МОТИВАЦИЯ

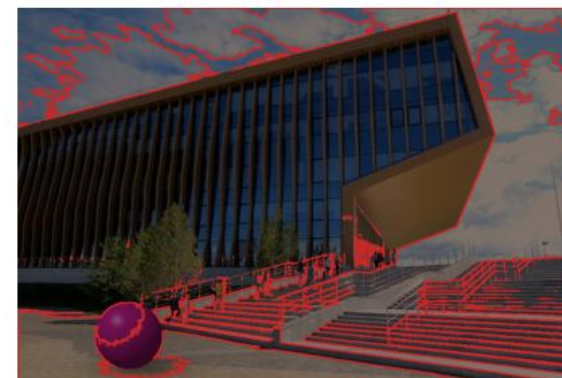
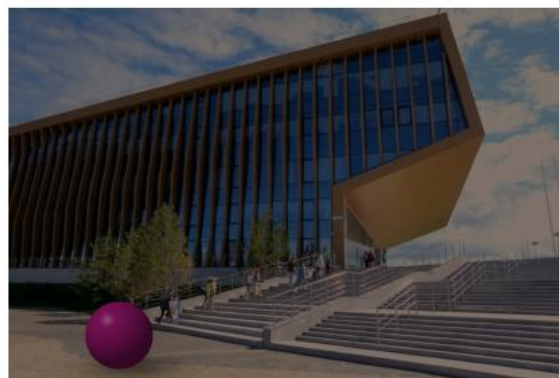
Дано: кадры «день» и «лампа». Найти: маску мяча в RGB, настроенную один раз при изменяющемся освещении

- Суть проблемы: Если настроить пороги цвета по кадру «день», то на кадре «лампа» этот же фильтр RGB найдет **всего 39% мяча** и ошибочно захватит **полфасада здания**.
- Фиксированные пороги в RGB «в лоб» не работают в реальных задачах (например, в робототехнике), так как цвет пикселей слишком сильно зависит от источника света.

кадр «день»: здесь настроили порог



кадр «лампа»: тот же мяч, другой свет порог в RGB с кадра «день»: найдено 39 % мяча



Один мяч при двух освещениях; порог в RGB, снятый с левого кадра, на правом разваливается

© фото — здание Университета Иннополис; собственная иллюстрация

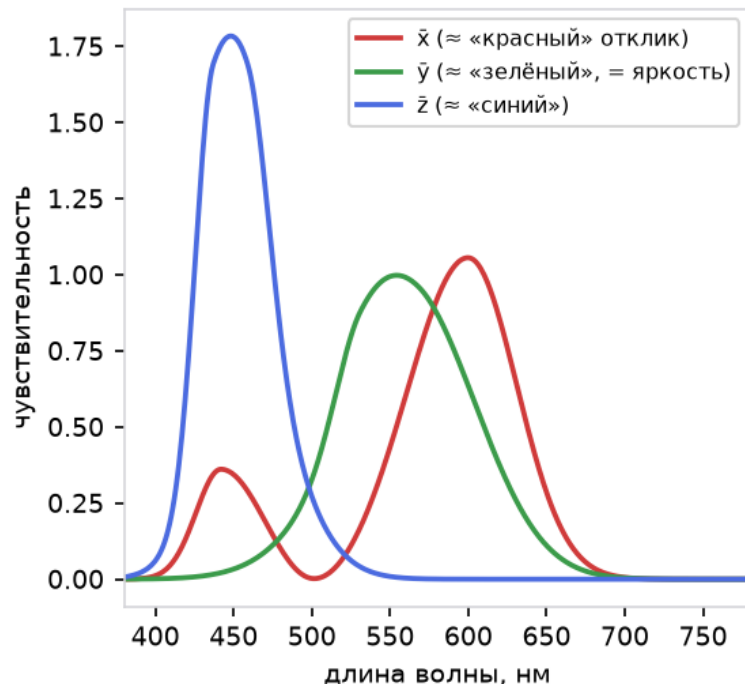
1

Цвет: от спектра к пространствам

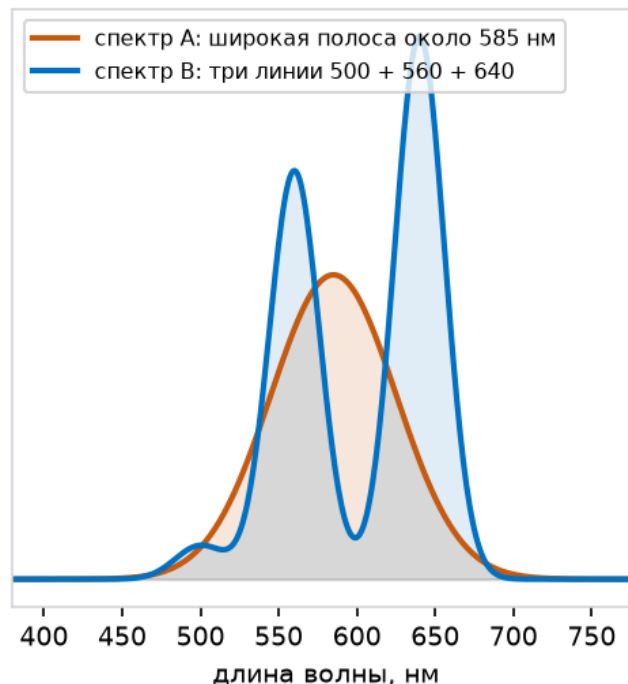
- Спектр → три числа; RGB, HSV, Lab, YCrCb; точечные операции, гистограмма, эквализация, CLAHE

Спектр → три числа: метамерия

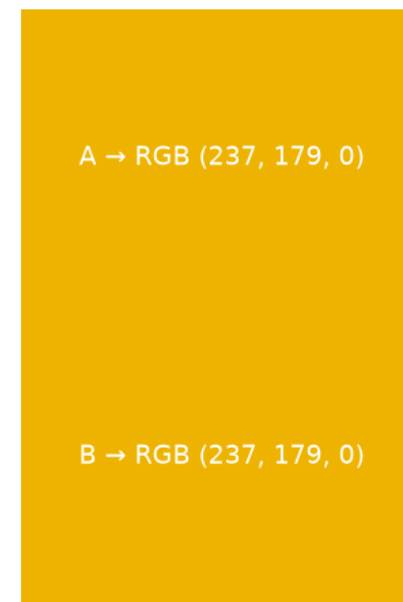
три кривые, три интеграла — и весь спектр
схлопывается в три числа (X, Y, Z)



два разных спектра (общего — 45 %)...



...один и тот же цвет: метамеры



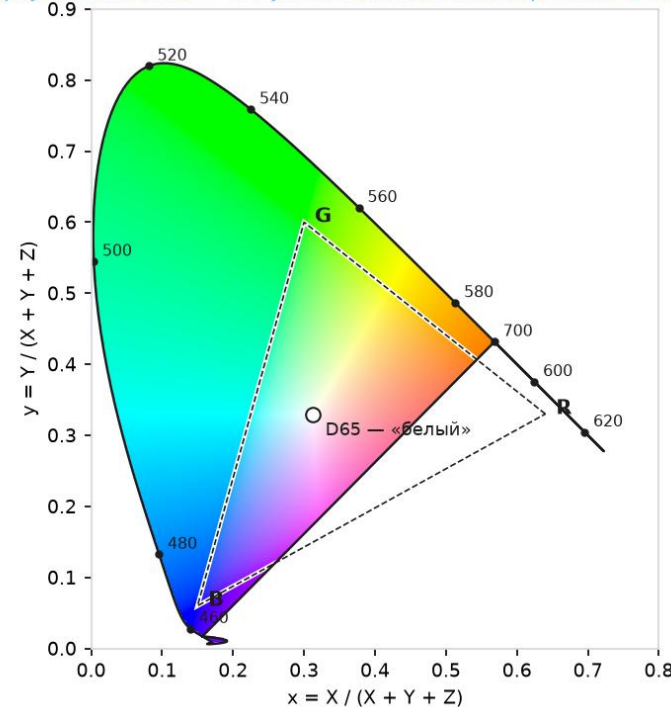
Сетчатка глаза и сенсор камеры не воспринимают весь спектр целиком. Они интегрируют (схлопывают) его с помощью трех кривых чувствительности в **три числа (X, Y, Z или RGB)**. **Физическая разница:** Спектр А (сплошная оранжевая полоса) и Спектр В (три узких синих пика) физически имеют мало общего. Проходя через датчики, оба этих совершенно разных спектра дают **один и тот же RGB-код (237, 179, 0)**. **Вывод:** Такие визуально идентичные, но физически разные цвета называются **метамерами**. Никакой RGB-код не способен их различить.

© собственная иллюстрация; кривые CIE 1931 — аппроксимация Wyman, Sloan, Shirley (2013)

Диаграмма цветности CIE 1931

Объясним устройство
диаграммы цветности CIE
1931 и ограничения
стандартных мониторов при
отображении цветов

Диаграмма цветности CIE 1931: подкова — все видимые цветности, треугольник sRGB — что умеет показать монитор: меньше половины



- **Ограничение мониторов (Треугольник sRGB):** Белый треугольник внутри подковы показывает цветовой охват стандартных экранов. Мониторы способны отобразить меньше половины видимых глазом цветов. Любой пиксель в формате RGB существует строго внутри этого треугольника.
- **Белая точка D65:** Центр треугольника — стандартная точка белого цвета, соответствующая естественному дневному освещению.

- **Переход к цветности:** Трехмерные координаты цвета (X, Y, Z) нормируют на их сумму, получая двумерную плоскость (x, y), где закодирован чистый цвет без учета яркости.
- **Человеческое зрение:** Вся цветная фигура в форме подковы представляет собой абсолютно все цвета, которые способен различить человеческий глаз.

© собственная иллюстрация; кривые CIE 1931 — аппроксимация Wyman, Sloan, Shirley (2013)

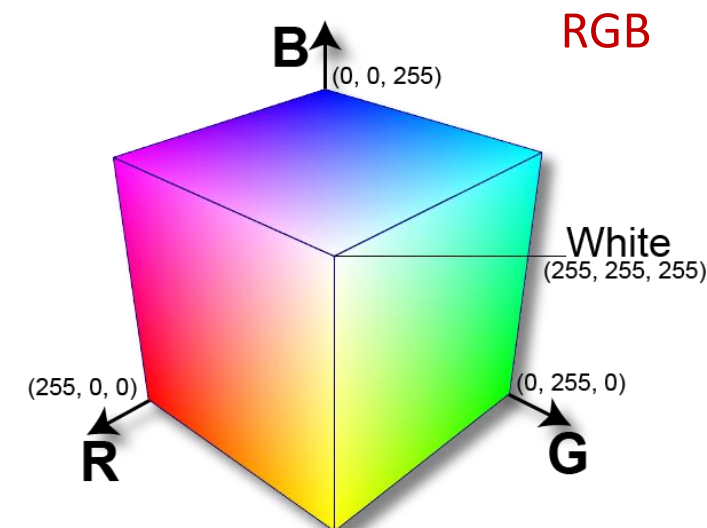
Сформулируем **математическую постановку задачи** по поиску устойчивого к свету описания цвета:

- **Проблема (Дано):** Исходные пиксели (R, G, B) сильно зависят одновременно и от самого объекта, и от условий освещения.
- **Цель (Найти):** Найти такое цветовое пространство, где **собственный цвет объекта и интенсивность света разделены** по разным осям/координатам.
- **Оценка качества (Критерий):** Какая доля пикселей целевого объекта успешно распознаётся маской при изменении освещения (на примере перехода «день» → «лампа»)

Смена цветового пространства — это просто **обратимая замена координат**. Она не добавляет новой информации в кадр, но кардинально меняет удобство её извлечения алгоритмами.

Физические свойства модели RGB и её главные недостатки для задач сегментации объектов:

- **Аддитивность:** Цвет формируется сложением трех световых потоков (так устроены матрицы камер и экраны).
- **Сильная корреляция каналов:** Значения R, G и B в реальных кадрах очень сильно связаны между собой (коэффициенты корреляции доходят до **0.95**).
- **Влияние освещения:** Изменение яркости (например, тень) **умножает все три канала сразу**. Тень — это не другой цвет, а просто укороченный вектор исходного цвета в пространстве RGB.
- **Специфика OpenCV:** Библиотека по умолчанию считывает каналы в обратном порядке — **BGR**. При этом евклидово расстояние между цветами в RGB не соответствует человеческому восприятию.



Модель RGB идеальна для хранения и вывода изображений, но плоха для цветовой фильтрации (порогов), поскольку в ней невозможно отделить чистый «цвет» от интенсивности «света».

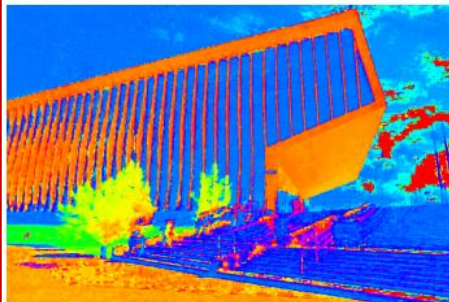
© fig — The RGB Color Space. Image taken from *Kenneth Moreland* (2018).

Три пространства, одно фото

оригинал
(BGR в памяти, RGB на экране)



H — оттенок, угол 0...179
(в OpenCV — половина градусов)



S — насыщенность:
0 у серого, 255 у чистого цвета



V — яркость
= $\max(R, G, B)$



HSV

Lab

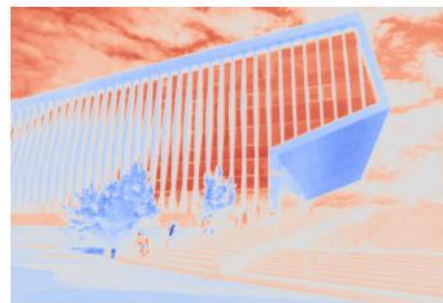
L — светлота (Lab):
зрительная, не физическая



a — зелёный ↔ пурпурный
(128 = нейтраль)



b — синий ↔ жёлтый
(128 = нейтраль)



Y (YCrCb)
= $0.299 R + 0.587 G + 0.114 B$



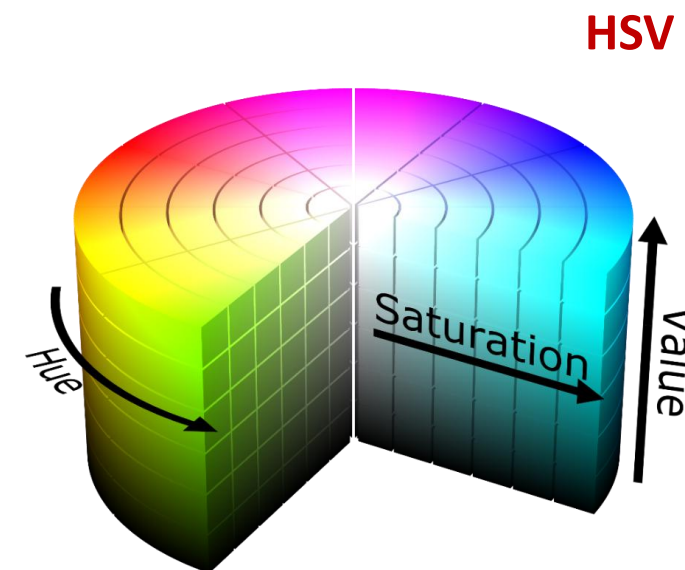
YCrCb

- **Цветовые каналы (H, a, b):** В каналах оттенка H (из HSV), а также a (зеленый-пурпурный) и b (синий-желтый) из Lab здание и небо четко разделены по цветам. При этом они практически **не реагируют на тени** и изменения освещения.
- **Яркостные каналы (V, L, Y):** Каналы яркости V (из HSV), светлоты L (из Lab) и Y (из YCrCb) содержат информацию только об освещенности, тенях и контрасте.

В пространствах HSV, Lab и YCrCb удалось успешно разнести «цвет» и «свет» по разным координатам. Это именно то свойство, которое искали для построения устойчивых к освещению масок.

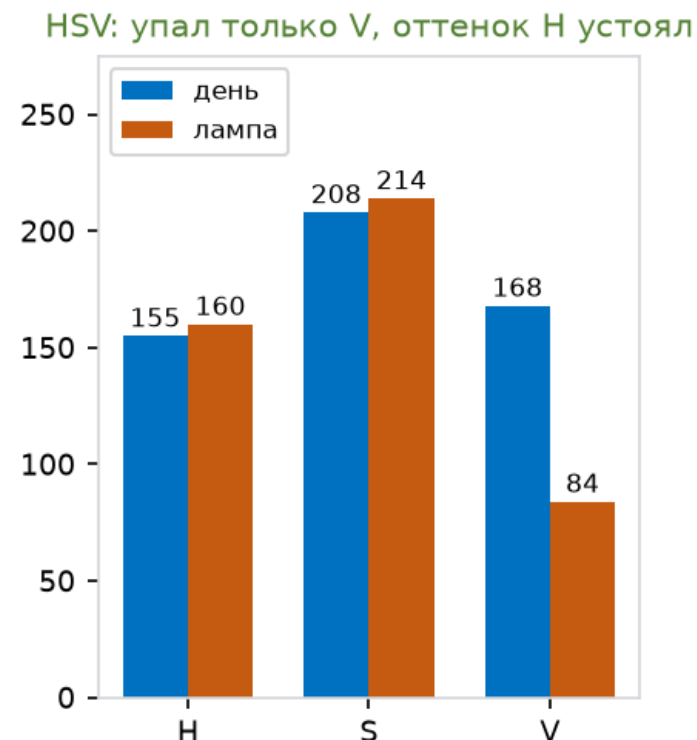
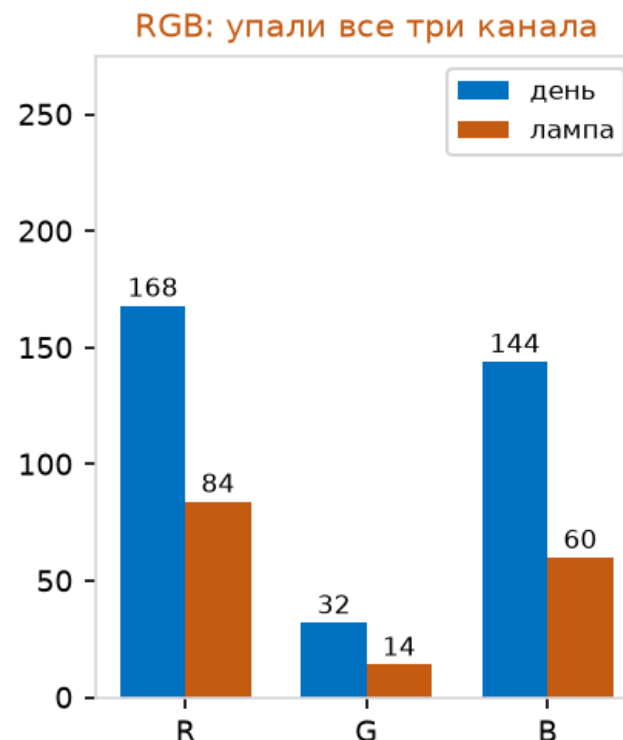
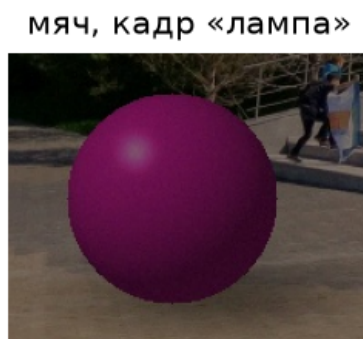
© фото — здание Университета Иннополис; собственная иллюстрация

- **Компоненты:** **H** (Hue) — оттенок в виде угла на цветовом круге; **S** (Saturation) — насыщенность (0 у серого); **V** (Value) — яркость, равная $\max(R, G, B)$.
- **Специфика OpenCV:** Чтобы угол оттенка поместился в тип данных `uint8` (0...255), его диапазон сжали с 0...360° до 0...179. Из-за этого стандартные диапазоны из интернета нужно обязательно делить на два.
- **Главный плюс:** Изменение света масштабирует только координату яркости (V падает со 168 до 84), но почти не меняет оттенок (H стабилен: 155 → 160).
- **Минусы («Цена»):** У серых цветов оттенок H становится случайным числом, а диапазон красного цвета разрывается, проходя через границу 0/179.



© fig — The HSV color wheel, with axes labeled. Image taken from Datumizer (2023).

Смена света: RGB против HSV



Преимущество пространства HSV перед RGB при изменении освещения. При смене освещения с кадра «день» на кадр «лампа» фиолетовый мяч становится темнее. Слайд сравнивает, что происходит с каналами:

- **В модели RGB рушится всё:** Падают значения абсолютно всех трех каналов (R, G, B);
- **В модели HSV оттенок стабилен:** Изменение света затронуло только яркость **V**, остальные показатели: H, S – стабильны.

Вывод: Маска, настроенная по координатам H и S, легко перенесет смену освещения, так как сам цвет (оттенок) математически отделился от интенсивности света.

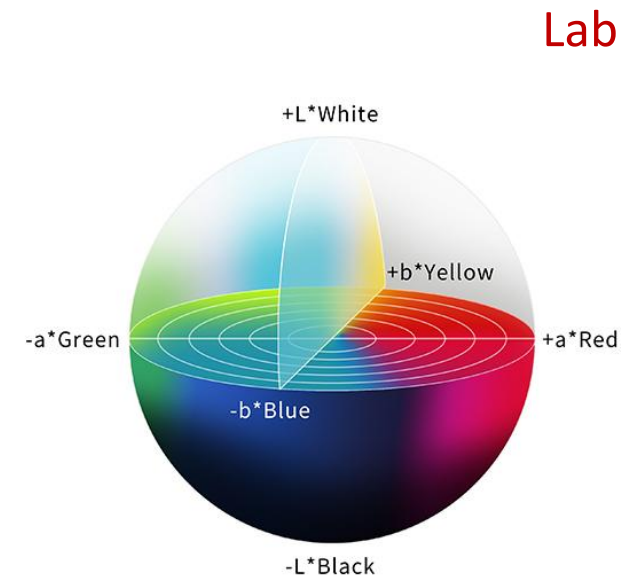
© фото — здание Университета Иннополис; собственная иллюстрация

Lab: расстояние между цветами

Устройство цветового пространства **Lab** (CIE $L^*a^*b^*$) и его главное преимущество — **перцептивная равномерность**

- **Компоненты:** L — светлота (яркость); a — ось от зелёного к пурпурному; b — ось от синего к жёлтому. Модель не зависит от устройства (аппаратно-независимая).
- **Перцептивная равномерность** (ΔE): Евклидово расстояние между точками в пространстве Lab строго соответствует тому, как разницу цветов воспринимает человек. Значение ($\Delta E \approx 2.3$) — это порог, когда человеческий глаз едва замечает различие.
- **Недостаток RGB на контрасте:** Одинаковый сдвиг на 40 кодов в RGB для синего цвета дает различие для глаза ($\Delta E = 6.4$), а для жёлтого — аж ($\Delta E = 14.1$). В RGB математическое расстояние не равно визуальному.
- **Специфика OpenCV:** Для упаковки в формат uint8 канал L растянут на диапазон 0...255, а цветовые оси a и b сдвинуты на +128.

Вывод: Пространство Lab идеально подходит для поиска **визуально похожих цветов**, эталонов и кластеризации, так как его метрика расстояния совпадает с человеческим зрением.



© fig — The CIE $L^*a^*b^*$ color space. Francesco Pistolesi (2018).

Устройство пространства **YCrCb** и формулирует золотое правило обработки яркости в компьютерном зрении

- **Разделение каналов:** Y — взвешенная яркость (luma); Chrominance Red — $Cr(R - Y)$ и Chrominance Blue — $Cb(B - Y)$ — цветоразностные сигналы, показывающие разность между цветовыми каналами и яркостью ($R - Y$ и $B - Y$). Е серого цвета строго равны нулю.
- **Человеческое зрение:** Формула яркости ($Y = 0.299R + 0.587G + 0.114B$) — это стандарт ITU-R BT.601, который используется для перевода цветного изображения в оттенки серого (или для выделения канала яркости Y). Коэффициенты при R , G , B — отражают биологическое устройство человеческого глаза (а не сиду светового потока). Она учитывает, что наш глаз гораздо чувствительнее к зелёному цвету, чем к синему (почти в 5 раз). Функция `cvtColor(BGR2GRAY)` в OpenCV считает черно-белое изображение ровно по этой формуле.
- **Применение:** Сжатие данных (например, JPEG хранит цветность в половинном разрешении) и обработка освещения без искажения исходных цветов.

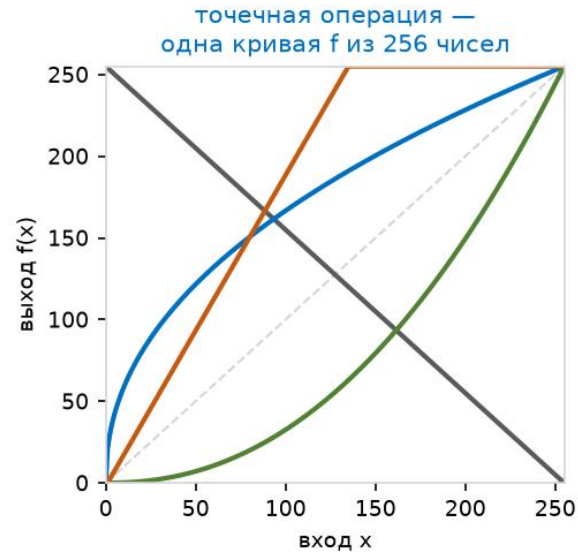
Главный закон курса: Любые операции улучшения кадра (эквализация, CLAHE, повышение резкости) нужно делать **строго по каналу Y (или V)**. Обрабатывать каналы R , G , B по отдельности **категорически нельзя**, так как это разрушит цвета.

Пространство	Когда работает (Плюсы)	Когда ломается (Минусы)
RGB / BGR	Хранение изображений, вывод на экран, арифметика каналов, нейросети	Порог по цвету при переменном свете, расчет евклидовых расстояний
HSV	Цветовые маски (<code>inRange</code>), трекары объектов, хромакей	Серые и темные пиксели (оттенок превращается в шум), разрыв красного на границе 0/179
Lab	Поиск «похожих цветов» (ΔE), кластеризация, сравнение с эталоном	Специфический uint8-масштаб в OpenCV, вычислительно дороже остальных
YCrCb / GRAY	Обработка яркости отдельно (эквализация, резкость), сжатие данных, выделение кожи	Цветовая маска по каналам Cr, Cb получается уже, чем по оттенку H

Вывод: Универсального цветового пространства не существует. Выбор модели полностью зависит от того, какую координату (яркость, визуальное сходство или чистый оттенок) вам нужно изолировать для решения задачи.

- **Определение:** Точечная операция — это преобразование $y = f(x)$, где новое значение пикселя зависит только от его собственного старого значения. Окрестности и соседние пиксели не учитываются.
- **Гамма-коррекция:** Нелинейное изменение яркости по одной общей кривой для всех каналов (например, код 128 превращается в 186 для компенсации особенностей человеческого зрения).
- **Баланс белого:** Применение своего отдельного множителя (коэффициента) к каждому цветовому каналу (например, умножение R на 1.15, а B на 1.12), чтобы скорректировать общее освещение кадра.
- **Главный плюс:** Без соседей нет сложных свёрток или размытий. Это быстрый алгоритм, который выполняется за один проход, а кривую функции $f(x)$ можно заранее рассчитать в виде таблицы LUT (Lookup Table) из 256 значений.

LUT (Lookup Table / Таблица поиска): 256 чисел вместо формулы



тёмный кадр (−2 EV):
коды только 1...135



`cv2.LUT(dark, stretch):`
та же информация, весь диапазон 0...255

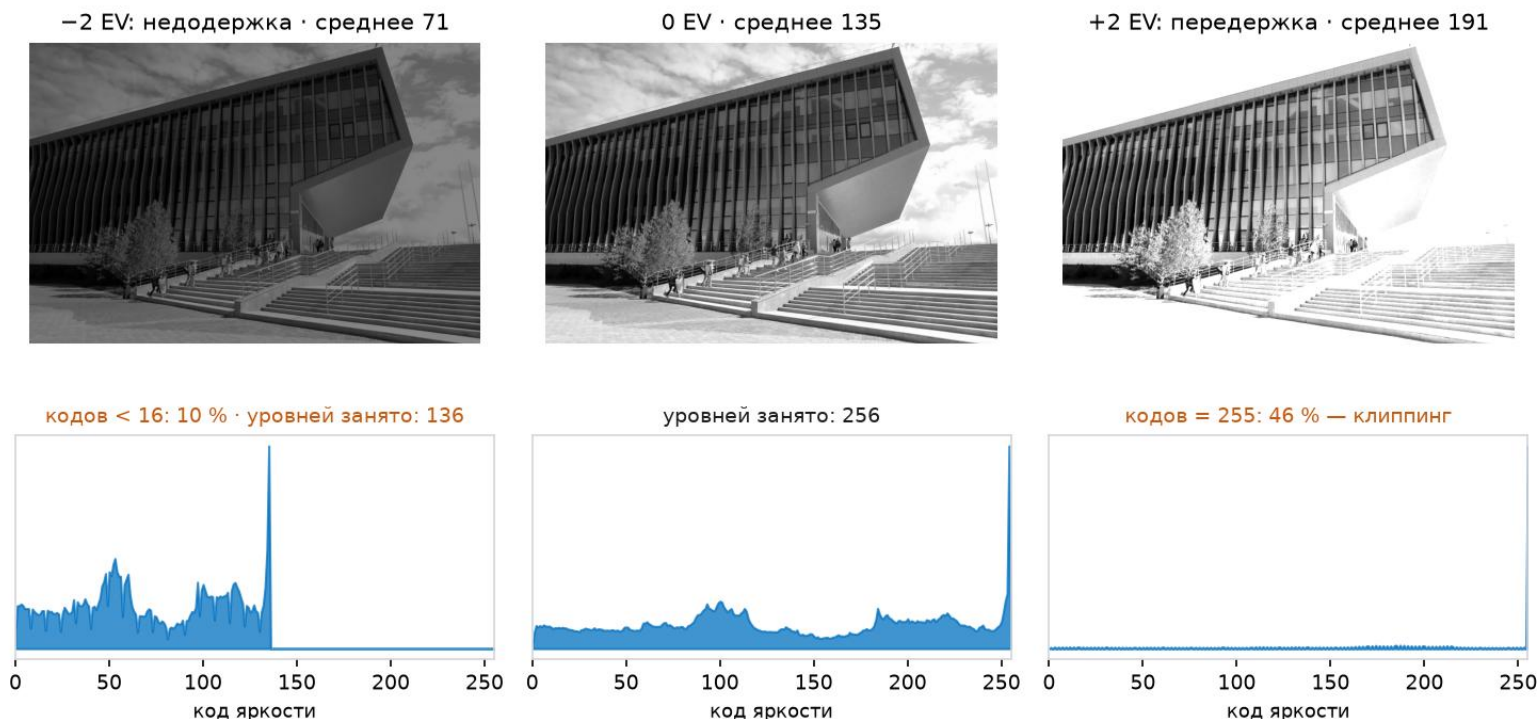


--- тождество — гамма-кодирование $x^{(1/2.2)}$: 128 → 186 — растяжение 1-99 %: [1, 135] → [0, 255]
— негатив: 255 − x — гамма-декодирование $x^{2.2}$

- **Идея оптимизации:** Вместо того чтобы вычислять формулу (возведение в степень для гаммы) для каждого из миллионов пикселей, функция $f(x)$ рассчитывается **всего один раз для 256 возможных значений** яркости (от 0 до 255) и сохраняется в массив.
- **Пример применения:** На примере тёмного кадра здания (коды занимают только узкий диапазон 1...135) показано растяжение яркости по процентиям 1–99% на весь диапазон 0...255 с помощью функции `cv2.LUT`.
- **Выигрыш в скорости:** Расчёт гаммы «в лоб» по формуле для кадра разрешением 1920×1279 занимает **126 мс**, а через готовую таблицу `cv2.LUT` — всего **1.8 мс** (выигрыш в скорости составляет два порядка).

© фото — здание Университета Иннополис; собственная иллюстрация

Гистограмма — паспорт кадра



- **Определение:**
Гистограмма — это график, который показывает количество (частоту) пикселей для каждого из 256 кодов яркости.

- **Три состояния кадра:**

- **Недосвет (-2 EV):** График смещен влево. 10% pxs темнее кода 16, занято всего 136 уровней яркости из 256. Картинка темная.
- **Норма (0 EV):** Идеальное распределение. Заняты все 256 уровней яркости, пики распределены равномерно по диапазону.
- **Пересвет (+2 EV):** График уходит вправо. **46% pxs в максимуме 255**, что вызывает **клиппинг** — потерю деталей в светлых зонах (пересветы).

© фото — здание Университета Иннополис; собственная иллюстрация

- **Метрики изображения:** **Экспозиция** определяется центром масс графика, **контраст** — шириной распределения (σ), а **клиппинг** (потеря деталей) виден как резкие пики на самых границах (в 0 или 255).
- **Поиск объектов:** Если на графике есть «две горки» (**бимодальное распределение**), это означает, что объект и фон легко разделить простым пороговым фильтром.
- **Правило контраста:** Растягивать яркость кадра нужно **по процентилям 1–99%**, а не по абсолютным min–max. Это защищает алгоритм от сбоев из-за одиночных "горячих" или битых пикселей.
- **Главное ограничение:** Гистограмма **не содержит пространственной информации** (не знает, где именно расположены пиксели). Совершенно разные по геометрии сцены могут иметь абсолютно одинаковую гистограмму.

Вывод: Гистограмма является «честным паспортом кадра». По ней алгоритм компьютерного зрения может автоматически понять, страдает ли изображение от шумов в тенях или от выбитых пересветов.

- **Цель алгоритма:** Найти такое преобразование $T(r)$, после которого коды яркости на выходе распределятся абсолютно равномерно $p_{out}(s) = \frac{1}{255}$ для всех s . Это максимизирует контраст кадра.
- **Математический вывод:** Из закона сохранения массы (количества пикселей) выводится, что искомая функция $T(r)$ — это просто **кумулятивная функция распределения** (интеграл гистограммы), умноженная на 255: $T(r) = 255 \cdot cdf(r)$.
- **Физический смысл:** Каждому исходному коду яркости выделяется тем большая доля выходного диапазона, чем больше пикселей имели этот код на исходной картинке. Популярные оттенки "растягиваются", редкие — "схлопываются".
- **Реализация:** Функция монотонна (порядок яркостей не нарушается), вычисляется мгновенно за $O(256)$ шагов по гистограмме и применяется ко всему кадру через **LUT-таблицу**. В OpenCV за это отвечает встроенная функция `cv2.equalizeHist`.

Эквализация: cdf становится прямой

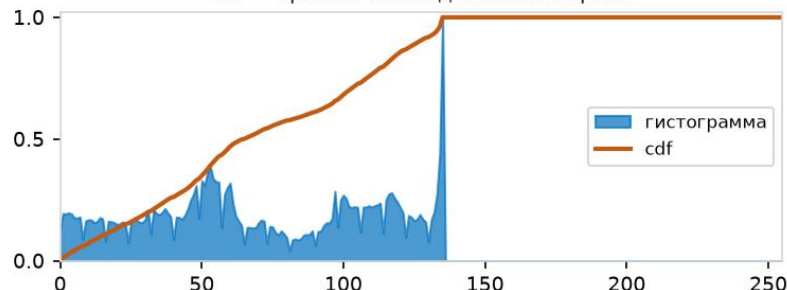
тёмный кадр: $\sigma = 40.6$



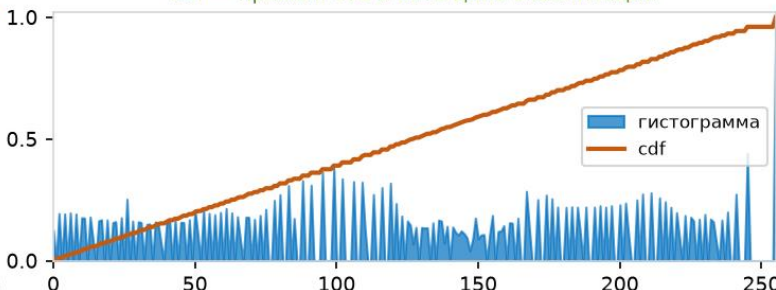
equalizeHist: $\sigma = 74.3$



cdf — кривая: все коды в левой трети



cdf — прямая: это и есть цель эквализации



Тёмный кадр: $\sigma = 40.6$, cdf — кривая, все коды в левой трети. После `equalizeHist` $\sigma = 74.3$, cdf — прямая. Но уровней не прибавилось: $136 \rightarrow 133$; гистограмма — гребёнка с зазорами, а наклон T в тенях 1.4 — во столько же вырос шум

Вывод: Эквализация не создаёт новую информацию, она лишь перераспределяет старую — неизбежно вытягивая и усиливая цифровой шум.

© фото — здание Университета Иннополис; собственная иллюстрация

тёмный цветной кадр (-2 EV)



equalizeHist по R, G, B отдельно:
оттенок уехал в среднем на 8.1°



equalizeHist только по Y (YCrCb):
оттенок сдвинулся на 1.2°



Эквализация по R, G, B отдельно строит три разные кривые — оттенок уезжает в среднем на 8.1° : фасад пожелтел, небо посинело. Та же операция по Y в YCrCb — сдвиг 1.2° . Правило: яркость отдельно, цвет не трогать

Вывод: не применяйте точечные операции изменения контраста к каналам RGB по отдельности. Всегда переводите кадр в пространства вроде YCrCb или HSV, обрабатывайте только яркость (Y или V) и возвращайте картинку обратно.

© фото — здание Университета Иннополис; собственная иллюстрация

CLANE: локально и с ограничением

Алгоритм **CLANE** (Contrast Limited Adaptive Histogram Equalization) решает проблемы классической глобальной эквализации

тень в углу: свет $1 \rightarrow 0.15$
 σ в тени 27.0
светлая часть: среднее 116



equalizeHist — одна кривая на кадр
 σ в тени 43.0
светлая часть: среднее 138



CLANE clip 2, плитки 8×8
 σ в тени 44.9
светлая часть: среднее 128



CLANE clip 40
тень «вытянута», но
мелкий шум усилен в 3.1 раза



Тень в углу кадра. equalizeHist одной кривой вытянул тень ($\sigma 27 \rightarrow 43$), но перекрасил и светлую часть (среднее $116 \rightarrow 139$). CLANE: своя кривая на каждую из 8×8 плиток, склейка билинейно — тень $\sigma 45$, светлая часть тронута меньше (129). Ограничение clipLimit держит шум: clip 40 усиливает мелкий шум в 3.1 раза

Вывод: CLANE — это мощный инструмент локального повышения контраста, который управляется двумя основными ручками: **размером плитки** и **порогом ограничения шума (cliplimit)**

© фото — здание Университета Иннополис; собственная иллюстрация

Параметр	Смысл	Типичное	Что если больше
equalizeHist — нет	одна кривая $T = 255 \cdot cdf$ на кадр	—	—
clipLimit	предел высоты столбца гистограммы плитки (в долях среднего)	2–4	контраст и шум растут: clip 40 → шум $\times 3.1$
tileGridSize	сетка плиток; кривая — на плитку, склейка билинейно	(8, 8)	локальнее; мелкая плитка «выравнивает» однородный объект в пятна
канал	к чему применять	Y или V, L	по R, G, B отдельно — оттенок уезжает на 8°

Сводная таблица-чеклист по настройке параметров глобальной эквализации и алгоритма CLAHE (за что отвечает каждая «ручка» настройки при улучшении контраста изображения, какие значения ставить по умолчанию и к чему приведут ошибки).

- **Суть цвета:** Цветовые пространства не создают новых данных, но эффективно разделяют «цвет» и «свет». Для масок используют **HSV**, для расчёта визуального сходства — **Lab**, а для работы с яркостью — **Y** (из **YCrCb**).
- **Инструменты коррекции:** Любую точечную операцию ускоряют через **таблицы LUT из 256 чисел**. Гистограмма служит «паспортом кадра», глобальная эквализация выпрямляет её кумулятивную функцию (cdf), а алгоритм CLAHE делает это локально и без критического усиления шума.
- **Главный пробел (Что дальше):** На данный момент изучена только подготовка изображений. Главная цель — **наложение порогов для превращения кадра в маску объекта** — будет разбираться во второй части занятия.

Демо 1.1 — три пространства, одно фото

python

```
1 import cv2, numpy as np
2 from cvcourse import io as cio
3 img = cio.imread("demo/iu-building.jpg")
4 b, g, r = [c.ravel().astype(np.float32) for c in cv2.split(img)]
5 cc = lambda p, q: np.corrcoef(p, q)[0, 1]
6 print("corr(R,G) = %.2f, corr(G,B) = %.2f, corr(R,B) = %.2f"
7       % (cc(r, g), cc(g, b), cc(r, b)))
8 hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
9 h, v = [hsv[..., i].ravel().astype(np.float32) for i in (0, 2)]
10 print("corr(H,V) = %.2f — оттенок не знает о яркости" % cc(h, v))
11 print("hue max = %d: в OpenCV H — половина градусов" % hsv[..., 0].max())
```

$\text{corr}(R,G) = 0.95$, $\text{corr}(G,B) = 0.93$, $\text{corr}(R,B) = 0.78$ · $\text{corr}(H,V) = 0.16$ — оттенок не знает о яркости · hue max = 179: в OpenCV H — половина градусов

Вызов

Что делает

`cv2.split(img)`

три канала B, G, R отдельными массивами — порядок OpenCV, не RGB

`np.corrcoef(p, q)[0, 1]`

коэффициент корреляции двух каналов: 1 — меняются вместе, 0 — независимо

`cv2.cvtColor(img, COLOR_BGR2HSV)`

та же картинка в координатах H, S, V; H в 0...179

Демо 1.2 — LUT: гамма за миллисекунды

python

```
1 import time
2 g = np.arange(256) / 255.0
3 lut = np.uint8(np.round(255 * g ** (1 / 2.2))) # гамма-кодирование
4 t0 = time.perf_counter()
5 fast = cv2.LUT(img, lut)
6 t1 = time.perf_counter()
7 slow = np.uint8(255 * (img / 255.0) ** (1 / 2.2))
8 t2 = time.perf_counter()
9 print("cv2.LUT: %.1f мс, np.power: %.0f мс — в %.0f раз дольше"
10       % (1e3 * (t1 - t0), 1e3 * (t2 - t1), (t2 - t1) / (t1 - t0)))
11 print("код 128 → %d: половина света — не половина кода" % lut[128])
12 print("совпадение до ±1 кода:", np.abs(fast.astype(int) - slow).max() <= 1)
```

cv2.LUT: 1.4 мс, np.power: 121 мс — в 86 раз
дольше · код 128 → 186: половина света —
не половина кода · совпадение до ±1 кода:
True

Вызов

Что делает

`np.uint8(np.round(255 * g ** (1/2.2)))`

таблица из 256 значений кривой — считается один раз, не на пиксель

`cv2.LUT(img, lut)`

подстановка по таблице для всех пикселей и каналов сразу

`np.uint8(255 * (img / 255.0) ** ...)`

то же по формуле: float на каждый пиксель — в сто раз дольше

Демо 1.3 — гистограмма и эквализация

```
python
1 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
2 lin = (gray / 255.0) ** 2.2 # код → свет
3 dark = np.uint8(255 * (0.25 * lin) ** (1 / 2.2)) # -2 EV → код
4 hist = cv2.calcHist([dark], [0], None, [256], [0, 256]).ravel()
5 print("кодов < 16: %.0f %, уровней занято: %d из 256"
6       % (100 * hist[:16].sum() / hist.sum(), (hist > 0).sum()))
7 eq = cv2.equalizeHist(dark)
8 print("σ до %.1f → после %.1f; уровней после: %d"
9       % (dark.std(), eq.std(), len(np.unique(eq))))
```

кодов < 16: 10 %, уровней занято: 136 из 256
· σ до 40.6 → после 74.3; уровней после: 133

Вызов	Что делает
$(gray / 255) ** 2.2 \rightarrow \times 0.25 \rightarrow ** (1/2.2)$	недодержка по-честному: в линейном свете, минус два стопа, обратно в код
<code>cv2.calcHist([dark], [0], None, [256], [0, 256])</code>	гистограмма: канал 0, без маски, 256 столбцов на диапазон 0...256
<code>cv2.equalizeHist(dark)</code>	LUT $T = 255 \cdot \text{cdf}$ по гистограмме кадра; только одноканальный uint8

Демо 1.4 — CLAHE против equalizeHist

python

```
1 h, w = gray.shape
2 x, x0 = np.arange(w, dtype=np.float32), 0.6 * w
3 light = np.where(x < x0, 1.0, 1 - 0.85 * (x - x0) / (w - 1 - x0))
4 uneven = np.uint8(gray * light.astype(np.float32)) # тень в правом углу
5 shadow = (slice(None), slice(w - 300, None)) # правые 300 px
6 eq = cv2.equalizeHist(uneven)
7 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
8 for name, im in (("вход", uneven), ("equalizeHist", eq),
9                 ("CLAHE 2.0", clahe.apply(uneven))):
10     print("%-13s σ в тени %5.1f · светлая часть: среднее %3.0f"
11           % (name, im[shadow].std(), im[:, :int(x0)].mean()))
```

вход: σ в тени 27.0 · светлая часть: среднее 116

equalizeHist: σ в тени 43.0 · светлая часть: среднее 139

CLAHE 2.0: σ в тени 44.9 · светлая часть: среднее 129

Вызов

Что делает

`np.where(x < x0, 1.0, ...)`

профиль света: 1 на левых 60 % кадра, дальше линейно до 0.15 — «тень в углу»

`cv2.createCLAHE(clipLimit, tileGridSize)`

объект CLAHE; `.apply(img)` — эквализация по плиткам с пределом высоты гистограммы

`im[shadow].std()`

контраст в тени: σ кодов в правых 300 столбцах

- **Вопрос 1.** Кадр эквализировали по R, G и B по отдельности. Останется ли объект «того же цвета» — да или нет, и почему?
- **Вопрос 2.** Объект занимает 2 % кадра, фон — плавный градиент света. Найдёт ли объект порог Otsu?

Ответы поднятием рук; разбор — во второй части, измерением

B1: нет — три разные кривые меняют соотношение каналов: оттенок уехал на 8°.

B2: нет — гистограмма одногорбая, Otsu разрежет фон пополам; нужен адаптивный порог или маска по цвету.

2

Пороги, морфология, компоненты

- От кадра к маске: глобальный порог, Otsu, адаптивный, `inRange`; от маски к объекту: морфология, связанные компоненты; за порогом — watershed, GrabCut, SLIC

- Дано: кадр — серый $I(x, y)$ или цветной; иногда эталонная маска для проверки
- Найти: **маску** $M(x, y) \in \{0, 255\}$ — «объект / не объект»; затем из маски — объекты: компоненты, bbox, центрост
- Критерий: доля верных пикселей против эталона; для bbox — **IoU** (пересечение на объединение)

Маска — самое дешёвое описание объекта: один бит на пиксель; всё дорогое начинается после неё

Бинарная маска — это самое вычислительно дешёвое описание объекта (всего один бит на пиксель). Все тяжелые и дорогие алгоритмы компьютерного зрения (контуры, трекинг, распознавание) запускаются строго после её построения.

- `cv2.threshold(gray, 128, 255, THRESH_BINARY)`: работает, пока свет ровный и гистограмма двугорбая
- На странице с градиентом света верных пикселей **58.3 %**: правая половина целиком «чернила»
- Число 128 взято ниоткуда — порог надо **вычислять из данных**: Otsu
- Пять режимов: `BINARY`, `BINARY_INV`, `TRUNC`, `TOZERO`, `TOZERO_INV` — все точечные операции, все одна LUT

Недостатки простейшего (наивного) метода бинаризации изображения фиксированным порогом на примере функции `cv2.threshold` с жестко заданным значением:

- Статический порог (например, `gray > 128`) работает хорошо только в идеальных условиях — когда освещение на снимке абсолютно ровное, а гистограмма имеет две четкие изолированные «горки».
- Сбой при градиенте света: Если на документе или сцене есть неравномерный свет (тень с одной стороны), этот метод полностью ломается.
- Проблема подбора: Число 128 берется наугад. Правильный порог нужно не угадывать, а динамически вычислять на основе анализа данных кадра (например, с помощью метода Otsu).

Ключевой шаг

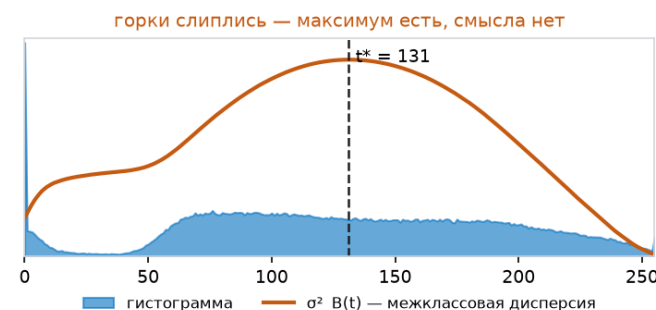
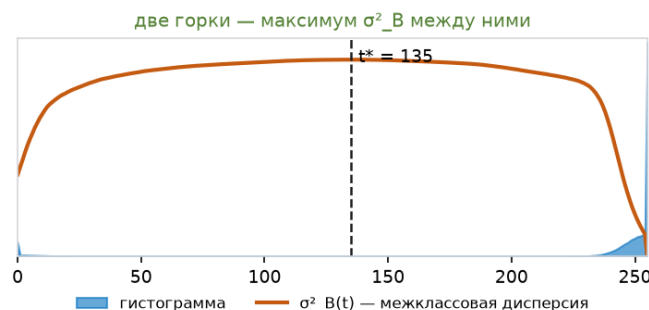
- Порог t делит гистограмму на два класса: доли w_0 , w_1 , средние μ_0 , μ_1
- Межклассовая дисперсия $\sigma^2_B(t) = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$; $t^* = \operatorname{argmax} \sigma^2_B(t)$ — перебор 256 значений
- **Что это значит физически:** ищем порог, при котором обе горки весомы и максимально далеки друг от друга
- Эквивалентно минимуму внутриклассовой: $\sigma^2_W + \sigma^2_B = \sigma^2$ — константа кадра
- `cv2.threshold(img, 0, 255, THRESH_BINARY + THRESH_OTSU)` → возвращает t^*

Алгоритм ищет порог, при котором «две горки» на гистограмме (фон и объект) оказываются одновременно весомыми (содержат много пикселей) и **максимально далеки друг от друга** по шкале яркости.

равномерный свет + шум → Otsu $t = 135$: верных пикселей 99.9 % градиент света + шум → Otsu $t = 131$: верных пикселей 56.8 %

Introduction to Computer Vision
Innopolis University, Fall 2026
Lecture 3: colour, histograms,
thresholds and morphology.
Otsu (1979): the threshold that
maximises between-class variance.
Adaptive: $T(x, y) = \text{local mean} - C$
CLAHE: clip limit 2, tiles 8×8

Introduction to Computer Vision
Innopolis University, Fall 2026
Lecture 3: colour, histograms,
thresholds and morphology.
Otsu (1979): the threshold that
maximises between-class variance.
Adaptive: $T(x, y) = \text{local mean} - C$
CLAHE: clip limit 2, tiles 8×8



Ровный свет: две горки, максимум $\sigma^2_B(t)$ между ними — $t^* = 135$, верных пикселей 99.9 %. Градиент света: горки слились в один горб, максимум σ^2_B формально есть — $t^* = 131$, но верных всего 56.8 %. Otsu оптимален для двугорбой гистограммы и бессилен для одногорбой

Метод Оцу разработан строго для двугорбых (бимодальных) гистограмм. Против световых градиентов и одногорбых распределений он бессилен. В таких условиях необходимо переходить к локальным методам — например, к адаптивному порогу (Adaptive Thresholding).

© собственная иллюстрация

- $T(x, y) = \text{mean}(\text{окно blockSize}) - C$ (MEAN_C); с гауссовыми весами — GAUSSIAN_C
- **Что физически:** порог следует за локальной яркостью; градиент света безразличен
- blockSize — нечётный, больше штриха и меньше пятна света: 11 → 99.0 %, 31 → 98.8 %, 91 → 98.0 %
- C — запас против шума на «пустой» бумаге: 5 → 87.5 %, 10 → 96.3 %, 15 → 98.8 %, 20 → 99.3 %
- Цена: внутри крупного однородного объекта порог = его яркость – C → **полосы и дыры**

Адаптивный порог (Adaptive Thresholding) решает проблему бинаризации при неравномерном освещении:

- Порог ($T(x, y)$) рассчитывается индивидуально для каждой точки как среднее (или гауссово взвешенное) значение яркости в скользящем окне blockSize за вычетом константы C. Физически порог «следует» за локальным светом, делая алгоритм устойчивым к градиентам освещения.
- Главный минус («Цена»): Внутри больших однородных объектов локальный порог становится равен их собственной яркости минус C. Это приводит к появлению артефактов в виде полос и дыр внутри объектов.

Четыре бинаризации на одном тексте

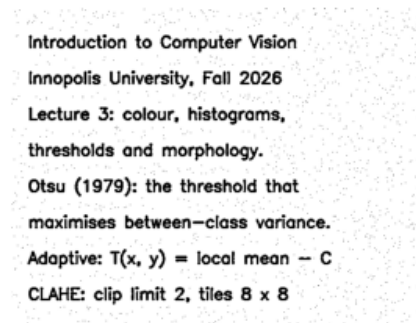
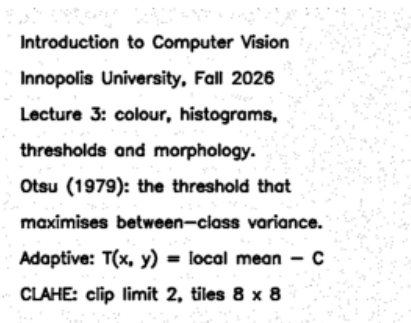
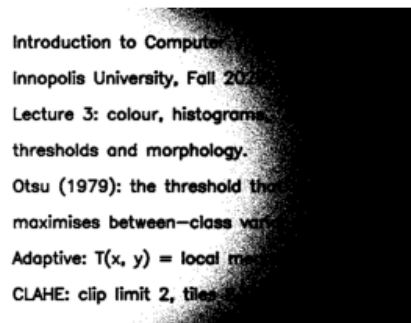
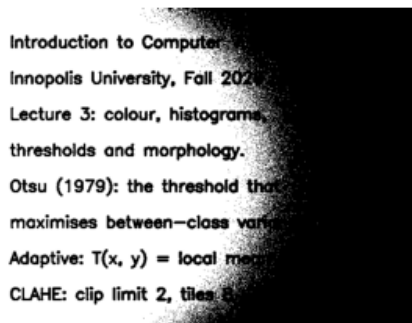
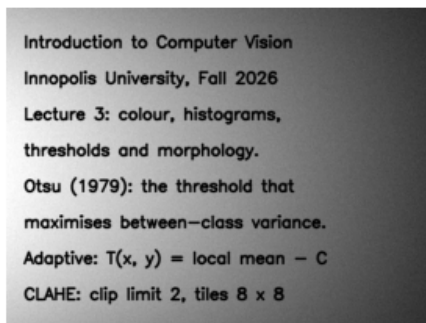
вход: свет 1 → 0.24 слева направо

порог 128
верных 58.3 %

Otsu, $t = 131$
верных 56.8 %

adaptive mean 31, $C = 15$
верных 98.8 %

adaptive gauss 31, $C = 15$
верных 99.1 %



Страница при свете 1 → 0.24 слева направо. Глобальный порог 128 — 58.3 % верных, Otsu — 56.8 %: правая половина «залита». Адаптивный (окно 31, $C = 15$): по среднему — 98.8 %, по Гауссу — 99.1 %. Это же сравнение соберём на занятии 3

Если на изображении присутствует неравномерное освещение:

- Глобальные методы полностью провалились: И жесткий порог 128, и автоматический метод Оцу (56.8% верных пикселей) не справились с тенью — правая половина кадра у них оказалась полностью «залита» черным цветом.
- Адаптивные методы работали отлично: За счет того, что порог рассчитывается локально для каждого окна, градиент света им безразличен. Адаптивный порог по среднему показал 98.8% точности, а по Гауссу — наилучший результат в 99.1%.

© собственная иллюстрация

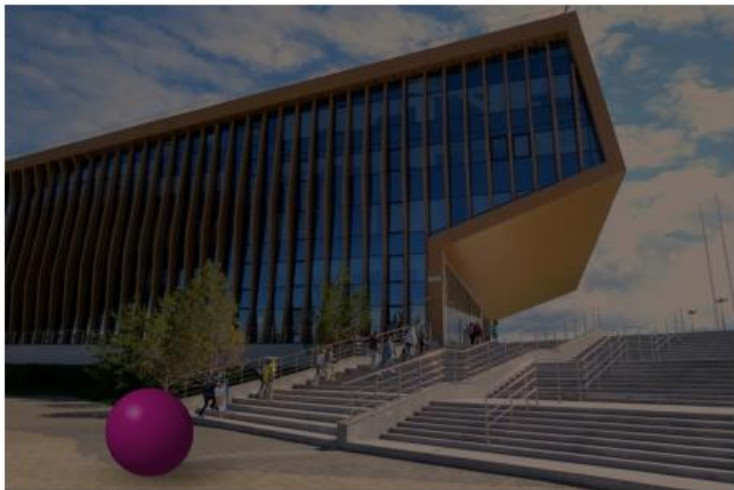
Правила построения цветовой маски с помощью функции `cv2.inRange` в пространстве HSV:

- `cv2.inRange(hsv, lo, hi)` → маска 0/255: все три канала в границах — параллелепипед в HSV
- Границы H и S — с эталона: 1–99 % пикселей объекта ± запас; V — только отсечь чёрное: $V \geq 40$
- Красный: две маски $[0, 10] \cup [170, 179]$ и `cv2.bitwise_or`; серое и тёмное — отсечь по S и V
- Диапазон живёт в **этой** камере: баланс белого сдвигает H — ★ занятия 3: подбор по клику
- Ограничение метода: Порог «живёт» только в условиях конкретной камеры. Смена баланса белого неизбежно сдвигает оттенок H.

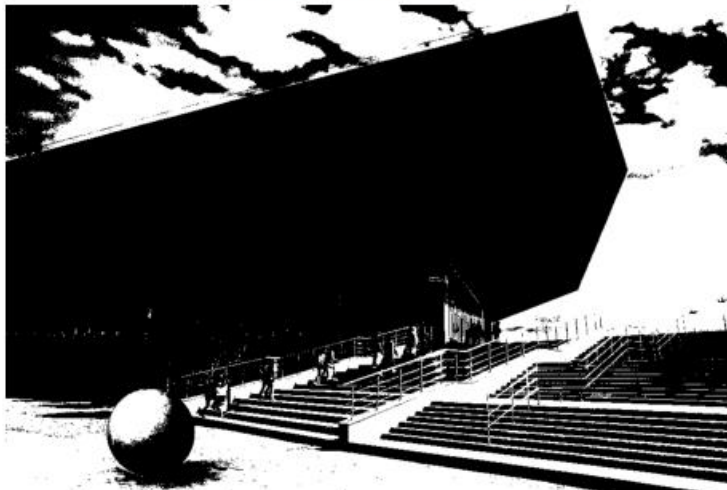
Вывод: `cv2.inRange` в HSV — основной рабочий инструмент цветовой сегментации, но он требует отсечения серого/тёмного шума (через S и V) и особой сборки для красных объектов.

inRange: RGB против HSV при смене света

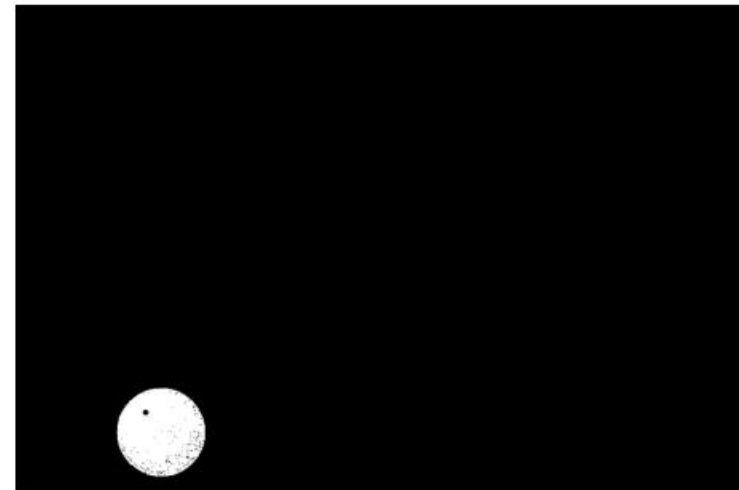
кадр «лампа»; границы сняты с кадра «день»
(1-99 % пикселей мяча $\pm$ 10 кодов)



RGB-параллелепипед: найдено 39 % мяча,
из найденного мяч — 2 %



HSV-параллелепипед: найдено 97 % мяча,
из найденного мяч — 100 %



Границы сняты с кадра «день» и применены к кадру «лампа». RGB-параллелепипед: найдено 39 % мяча, и лишь 2 % найденного — мяч. HSV с открытым V: найдено 97 %, из найденного мяч — 100 %. **Дыра в маске — блик: там S упала;** её зашьёт закрытие

Если пороги цвета были настроены по кадру «день», а затем применены к кадру «лампа»:

- Фильтрация в RGB полностью провалилась;
- Фильтрация в HSV сработала идеально: Маска с открытым порогом яркости V успешно нашла 97% мяча, и в неё не попало абсолютно ничего лишнего (100% точность попадания).

© фото — здание Университета Иннополис; собственная иллюстрация

Основы бинарной морфологии — базовых операций очистки маски от шумов с помощью движущегося окна-формы (структурирующего элемента)

- Работает с маской и **структурирующим элементом** (structuring element) — окном-формой: `getStructuringElement(MORPH_ELLIPSE, (5, 5))`
- **Эрозия** (erosion): пиксель остаётся, если элемент **целиком** внутри маски — крапинки и перемычки исчезают, объект худеет: 10 801 → **8 427** px
- **Дилатация** (dilation): пиксель включается, если элемент **задел** маску — дыры затягиваются, объект толстеет: → **17 137** px
- Порознь искажают размер; по очереди — чистят без искажения: открытие и закрытие

Суть двух главных операций:

- Эрозия (Erosion) — «сужение»: Пиксель маски сохраняется только в том случае, если структурирующий элемент целиком поместился внутри объекта;
- Дилатация (Dilation) — «расширение»: Пиксель включается, если структурирующий элемент хотя бы краем задел маску. Это позволяет затягивать дыры и блики внутри объекта, но при этом сам объект «толстеет».

Комбинированные морфологические операции — открытие и закрытие — идеально очищают бинарную маску от шумов, сохраняя исходный размер объекта.



Маска с 256 компонентами и 25 дырами. Эрозия убирает крапинки, но раздувает дыры; дилатация зашивает дыры, но раздувает крапинки. Открытие (эрозия → дилатация, 5 × 5): 1 компонента, размер прежний. Закрытие (дилатация → эрозия, 9 × 9): 0 дыр, размер прежний. Ядро — чуть больше помехи

- Исходная «грязная» маска: Содержит 256 ложных компонентов (мелкий шум вокруг) и 25 дыр (блики внутри мяча).
- Открытие (Opening = Эрозия → Дилатация): Сначала эрозия стирает мелкие крапинки, а затем дилатация возвращает границы мяча обратно. Результат: 1 чистый компонент, исходный размер сохранен, но дыры внутри остались.
- Закрытие (Closing = Дилатация → Эрозия): Дилатация сначала затягивает все внутренние дыры, а затем эрозия возвращает объекту прежний масштаб.

© собственная иллюстрация

Операция	Формула	Убирает / даёт	Ядро
открытие (opening)	эрозия $\rightarrow$ дилатация	крапинки, тонкие «усы»: 256 компонент $\rightarrow$ 1	чуть больше помехи: 5×5
заккрытие (closing)	дилатация $\rightarrow$ эрозия	дыры, разрывы контура: 25 дыр $\rightarrow$ 0	чуть больше дыры: 9×9
градиент (gradient)	дилатация – эрозия	контур маски толщиной в ядро	3×3
top-hat / black-hat	вход – открытие / закрытие – вход	яркие / тёмные мелочи на неровном фоне	больше детали, меньше фона

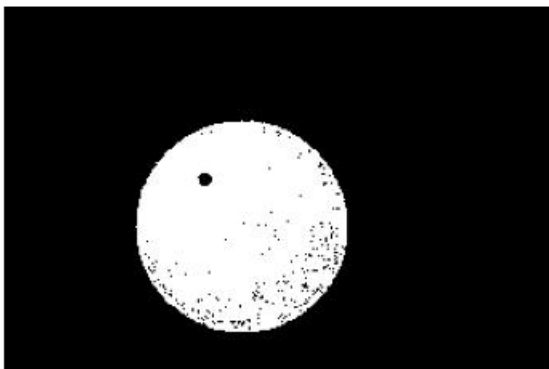
Поиск связных компонент превращает сырой набор пикселей бинарной маски в конкретные оцифрованные объекты

- Маска — множество пикселей; **компонента** — те, что связаны соседством: **4-связность** (ребро) или **8** (и по диагонали)
- `connectedComponentsWithStats(mask)` → число, карта меток, stats (x, y, w, h, area на компоненту), centroids
- $O(N)$: на 960×640 — доли миллисекунды
- Дальше — фильтр по площади: крупнейшая → объект; мелкие → мусор; несколько крупных → несколько объектов
- Слипшиеся объекты — **одна** компонента: разделит watershed, не морфология
- **Быстрое извлечение данных:** Функция OpenCV за один проход алгоритма $O(N)$ за доли миллисекунды) находит все группы пикселей и возвращает для каждой из них: координаты рамки (x, y, w, h), площадь (area) и центр масс (centroids).
- **Фильтрация шума по площади:** На основе статистики area легко отделить полезный объект от грязи. Крупнейшая компонента принимается за искомый объект, а мелкие — удаляются как мусор.
- **Главное ограничение:** Если два объекта на кадре соприкоснулись (слиплись), алгоритм посчитает их за одну компоненту. Морфология здесь бессильна, и для их разделения нужно использовать алгоритм водораздела (Watershed).

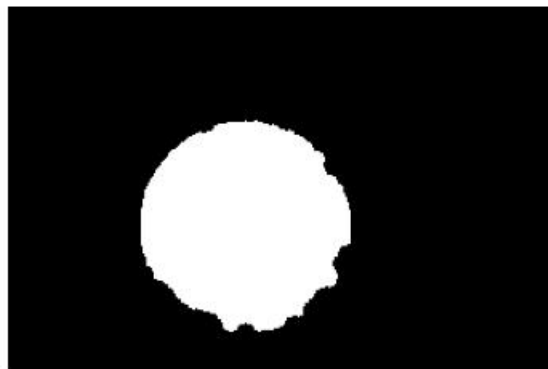
Конвейер трекера: от маски к bbox

Сквозной конвейер (pipeline) трекера объектов

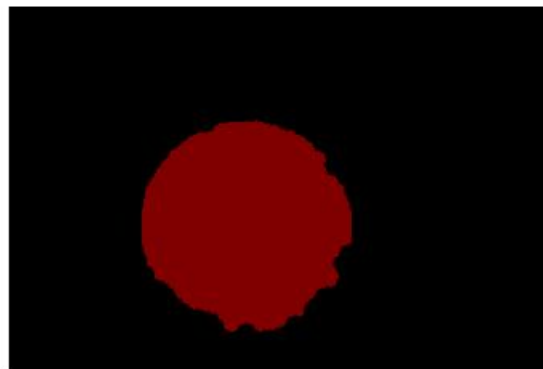
1. `inRange` в HSV:
блик и тень — дыры и рваный край



2. открытие 5×5 + закрытие 9×9 :
1 компонента



3. `connectedComponentsWithStats`:
метки, площади, bbox, центроиды



4. крупнейшая компонента → bbox
 115×116 , площадь 9996 px



Кадр «лампа»: `inRange` в HSV → открытие 5×5 и закрытие 9×9 → одна компонента → крупнейшая → bbox 115×116 , площадь 9996 px, центроид (190, 558). IoU с эталонным прямоугольником 97 %. Это весь трекер занятия 3 — четыре вызова на кадр

- Цветовой фильтр (`inRange` в HSV):** Создание первичной маски по порогам цвета кадра «день». На кадре «лампа» она получается с дырой от блика и рваными краями.
- Морфологическая очистка (Открытие + Закрытие):** Фильтрация окном 5×5 стирает фоновый шум, а окно 9×9 полностью затягивает дыру от блика. На выходе остается ровно одна чистая компонента.
- Анализ связных областей :** Алгоритм маркирует объект, вычисляет его площадь, координаты рамки и центр масс (центроид).
- Выделение объекта (bbox):** Выбирается крупнейшая найденная компонента. На кадре рисуется желтая рамка размера 115×116 пикселей с площадью 9996 px. Точность поиска по метрике IoU составляет 97%.

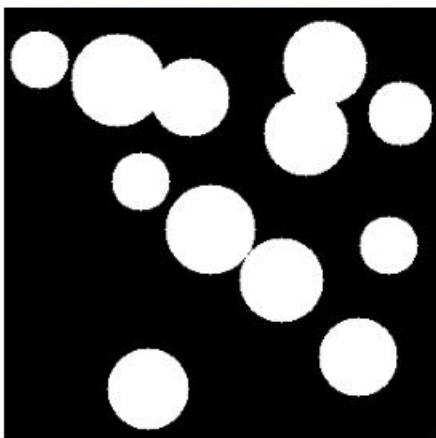
© фото — здание Университета Иннополис; собственная иллюстрация

- **Watershed** (водораздел): рельеф «заливают» от маркеров — пиков `distanceTransform`; границы — где потоки встречаются; слипшиеся диски: 9 компонент → **12**
 - **GrabCut** (Rother, Kolmogorov, Blake, 2004): прямоугольник → две смеси гауссиан «объект/фон» + разрез графа; 5 итераций, IoU **99.8 %**, ≈ 80 мс
 - **SLIC** (Achanta et al., 2012): k-means в (L, a, b, x, y) → **суперпиксели**: 187 однородных кусочков за ≈ 50 мс — единицы вместо пикселей
 - В 2026: предобработка и разметка в ML-пайплайнах, когда цвета и порога мало
-
- **Алгоритм водораздела (Watershed)**: Используется для разделения слипшихся объектов известной формы. Обычный поиск связанных компонентов видит вместо 12 дисков всего 9 контуров. Watershed, отталкиваясь от пиков преобразования расстояний (`distanceTransform`), безошибочно разделяет их обратно на 12 отдельных объектов.
 - **Интерактивная сегментация (GrabCut)**: Идеальна для выделения одного сложного объекта по грубой рамке. Алгоритм на основе графов за 5 итераций (80 мс) идеально отделил фиолетовый мяч от фона, полностью отбросив падающую тень и показав точность IoU 99.8%.
 - **Суперпиксели (SLIC)**: Применяются для группировки пикселей в однородные по цвету и текстуре кластеры (суперпиксели). Алгоритм за 47 мс разбил кадр на 187 элементов, чьи границы четко легли по краям окон, фасада здания и неба. Это радикально уменьшает объем данных перед подачей в тяжелые нейросети.

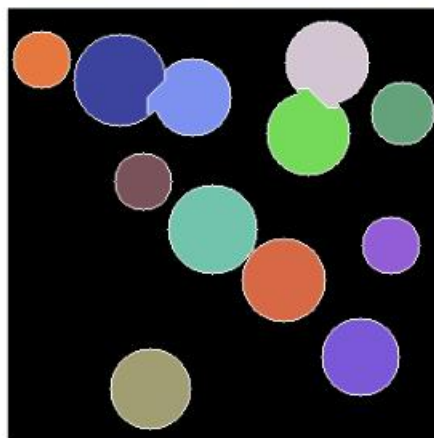
Три способа резать кадр

Три продвинутых метода сегментации изображений, которые применяются, когда обычных цветовых и яркостных порогов недостаточно.

12 дисков слиплись:
связных компонент — 9



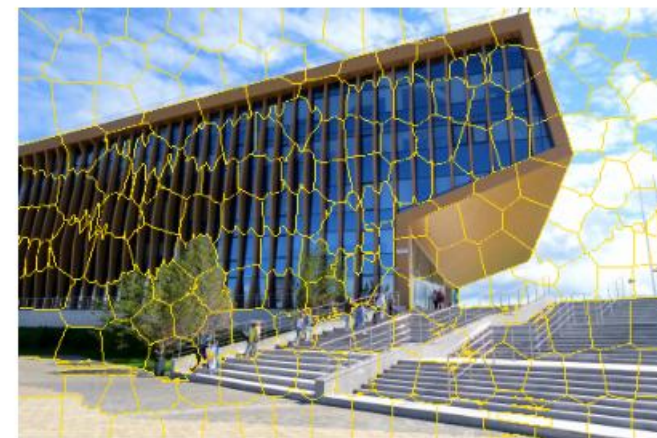
watershed от маркеров
(пики distanceTransform): 12



GrabCut по прямоугольнику, 5 итераций,
80 мс: IoU с эталоном 99.8 %



SLIC: 187 суперпикселей за 47 мс —
единицы крупнее пикселя, но однородные



Слева: 12 слипшихся дисков — компонент 9, watershed от пиков distanceTransform — 12. В центре: GrabCut по прямоугольнику за 5 итераций — маска мяча с IoU 99.8 %, тень отброшена. Справа: 187 суперпикселей SLIC — границы легли по краям фасада, окон и неба

- **Алгоритм водораздела (Watershed):** Используется для разделения слипшихся объектов схожей формы.
- **Интерактивная сегментация (GrabCut по прямоугольнику):** Идеальна для выделения одного сложного объекта по грубой рамке.
- **Суперпиксели (SLIC):** Применяются для группировки пикселей в однородные по цвету и текстуре кластеры (суперпиксели).

© фото — здание Университета Иннополис; собственная иллюстрация

Робототехника

- маркер, ворота, конус: `inRange` → морфология → компонента; 30 FPS без GPU
- LUT и порог — реальное время: 1–2 мс на кадр
- враг — освещение: HSV, V открыт, границы калибровать на месте

МОИИ

- маска по цвету, GrabCut, суперпиксели — дешёвая разметка перед моделью
- гистограмма и CLAHE — аугментации и нормализация; но по Y, не по каналам
- пороги и компоненты — постобработка сегментатора: убрать крапинки, оставить крупнейшую

- Порог — из данных, не из головы: Otsu для двугорбой гистограммы, адаптивный при неровном свете, `inRange` по H и S — по цвету
- Маска чистится открытием и закрытием без потери размера; объект — крупнейшая связная компонента с `bbox` и центроидом
- Открытый вопрос: маска знает «где», но не «какой формы» — контуры и моменты в **L7**, слипшиеся объекты и движение — L13–L14

Демо 2.1 — четыре бинаризации текста

python

```
1 from cvcourse import io as cio
2 gt = cio.imread("demo/text-gt.png", "gray") # чернила 0, бумага 255
3 page = cio.imread("demo/text-lamp.png", "gray") # свет 1 → 0.24 слева направо
4 t, otsu = cv2.threshold(page, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
5 res = [("порог 128", cv2.threshold(page, 128, 255, cv2.THRESH_BINARY)[1]),
6        ("Otsu, t = %d" % t, otsu)]
7 for name, flag in (("adaptive mean", cv2.ADAPTIVE_THRESH_MEAN_C),
8                    ("adaptive gauss", cv2.ADAPTIVE_THRESH_GAUSSIAN_C)):
9     res.append((name, cv2.adaptiveThreshold(
10         page, 255, flag, cv2.THRESH_BINARY, 31, 15)))
11 for name, b in res:
12     print("%-15s верных пикселей %.1f %" % (name, 100 * (b == gt).mean()))
```

порог 128 — 58.3 % · Otsu, t = 131 — 56.8 % ·
adaptive mean — 98.8 % · adaptive gauss —
99.1 % верных пикселей

Вызов

`cv2.threshold(page, 0, 255, BINARY + OTSU)`

`cv2.adaptiveThreshold(page, 255, flag, BINARY, 31, 15)`

`(b == gt).mean()`

Что делает

порог по Otsu: своё «0» игнорируется, найденный t возвращается первым

порог в каждой точке = среднее (или гауссово) по окну 31 минус C = 15

доля пикселей, совпавших с эталоном, — метрика бинаризации одним выражением

Демо 2.2 — маска → морфология → bbox

python

```
1 frame = cv2.imread("demo/ball-lamp.jpg")
2 hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
3 mask = cv2.inRange(hsv, (140, 121, 40), (168, 230, 255)) # H, S — с кадра «день»
4 k5 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
5 k9 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
6 mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k5) # крапинки прочь
7 mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k9) # дыры (блик) зашить
8 n, labels, stats, cent = cv2.connectedComponentsWithStats(mask)
9 i = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
10 x, y, w, h, area = stats[i]
11 print("компонент: %d; bbox (%d, %d, %d × %d), площадь %d px"
12       % (n - 1, x, y, w, h, area))
```

компонент: 1; bbox (133, 502, 115 × 116),
площадь 9996 px

Вызов

`cv2.inRange(hsv, (140, 121, 40), (168, 230, 255))`

`cv2.morphologyEx(mask, MORPH_OPEN, k5)`

`stats[1:, cv2.CC_STAT_AREA]`

Что делает

маска 0/255: H 140...168, S 121...230 — с кадра «день»; V от 40 — только отсечь чёрное

открытие эллипсом 5 × 5: крапинки; затем MORPH_CLOSE 9 × 9: дыры

площади компонент без фона (метка 0); `cent[i]` — центр масс для трека

Демо 2.3 — крапинки, дыры и компоненты

python

```
1 noisy = cv2.imread("demo/mask-noisy.png", "gray") # маска + крапинки + дыры
2 holes = lambda m: cv2.connectedComponents(255 - m)[0] - 2 # фон минус внешний
3 n0 = cv2.connectedComponentsWithStats(noisy)[0] - 1
4 opened = cv2.morphologyEx(noisy, cv2.MORPH_OPEN, k5) # ядра — из демо 2.2
5 closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, k9)
6 print("компонент: %d → %d после открытия 5 × 5"
7       % (n0, cv2.connectedComponentsWithStats(opened)[0] - 1))
8 print("дыр: %d → %d после закрытия 9 × 9" % (holes(noisy), holes(closed)))
9 print("площадь: %d → %d px (одна эрозия оставила бы %d)"
10       % ((noisy > 0).sum(), (closed > 0).sum(), (cv2.erode(noisy, k5) > 0).sum()))
```

компонент: 256 → 1 после открытия 5 × 5
дыр: 25 → 0 после закрытия 9 × 9
площадь: 10801 → 10554 px (одна эрозия оставила бы 8427)

Вызов

`cv2.connectedComponents(255 - m)[0] - 2`

`cv2.connectedComponentsWithStats(m)[0] - 1`

`cv2.erode(noisy, k5)`

Что делает

дыры = компоненты **инверсии** маски минус внешний фон и минус метка 0

число компонент маски без фона

одна эрозия: крапинок нет, но площадь −22 % — поэтому открытие, а не эрозия

Демо 2.4 — где ломается hue

```
1  rng = np.random.default_rng(0)
2  grey = rng.normal(128, 4, (100, 100, 3))          # серый + шум  $\sigma = 4$ 
3  h = cv2.cvtColor(np.uint8(np.clip(grey, 0, 255)), cv2.COLOR_BGR2HSV)[..., 0]
4  print("серый патч: hue от %d до %d,  $\sigma = %.0f$  — у серого нет оттенка"
5        % (h.min(), h.max(), h.std()))
6  red = rng.normal((30, 30, 220), 8, (100, 100, 3))  # красный + шум  $\sigma = 8$ 
7  h = cv2.cvtColor(np.uint8(np.clip(red, 0, 255)), cv2.COLOR_BGR2HSV)[..., 0]
8  print("красный: hue < 10 у %.0f %% пикселей, hue > 170 у %.0f %%"
9        % (100 * (h < 10).mean(), 100 * (h > 170).mean()))
```

python

серый патч: hue от 0 до 178, $\sigma = 52$ — у серого нет оттенка · красный: hue < 10 у 62 % пикселей, hue > 170 у 38 %

Вызов

`rng.normal(128, 4, (100, 100, 3))`

`np.uint8(np.clip(..., 0, 255))`

`(h < 10).mean(), (h > 170).mean()`

Что делает

серый патч с шумом камеры $\sigma = 4$: каналы почти равны, hue — деление 0 на 0

правило курса: в байт только через обрезку — иначе переполнение

доли красного по обе стороны разрыва 0/179

Фрагмент «найти красный конус»

- `hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`
- `mask = cv2.inRange(hsv, (0, 0, 0), (10, 255, 255))`
- `x, y, w, h = cv2.boundingRect(mask)`

Три ошибки

- красный рвётся через 0/179: нужна вторая маска 170...179 — иначе **38 %** конуса потеряно
- $S \geq 0, V \geq 0$: серый асфальт и тени с $\text{hue} \approx 0$ попадут в маску
- `boundingRect` всей маски обнимет и мусор: сначала открытие и крупнейшая компонента

ПОДВОДНЫЕ КАМНИ

- **H в OpenCV — 0...179**, не 0...359: диапазон из статьи делить на два; красный — две маски через 0/179
- **Hue у серого и тёмного — шум**: без порогов по S и V маска «цветёт» на асфальте и в тенях
- **Границы по V не переносятся**: свет меняет V; с эталона брать H и S, V — только отсечь чёрное
- **`cvtColor` ждёт `uint8` или `float32` в 0...1**: float с кодами 0...255 даёт мусор в H молча

ПОДВОДНЫЕ КАМНИ

- **Эквализация по R, G, B отдельно** сдвигает оттенок — 8° на нашем фото: только по Y, V или L
- **`blockSize` чётный** → ошибка; слишком мелкий → полосы и дыры внутри крупных объектов; $C \approx 10-20$ против шума
- **Ядро больше объекта**: открытие удалит объект вместе с крапинками, закрытие слепит соседей
- **Маска `bool` или 0/1 вместо 0/255**: `connectedComponents` хочет `uint8`, а `mask.sum()` в 0/255 — не число пикселей

KEY TAKEAWAYS

- Цвет — три интеграла, не спектр: метамеры дают один RGB; каналы RGB коррелированы на 0.95 и все зависят от света — порог в RGB с «дня» нашёл на «лампе» **39 %** мяча, порог по H и S — **97 %**
- Точечная операция — LUT из 256 чисел, в ~100 раз быстрее формулы; гистограмма — паспорт кадра: клиппинг 46 % при +2 EV, 136 уровней при -2 EV
- Эквиализация $T = 255 \cdot \text{cdf}$: σ 40.6 $\rightarrow$ 74.3, но уровней не прибавляет, шум в тенях $\times 1.4$, по каналам сдвигает оттенок на 8° ; CLAHE (clip 2, 8×8) — локально и с пределом на шум
- Otsu — максимум межклассовой дисперсии: 99.9 % при ровном свете и 56.8 % при градиенте; тогда `adaptiveThreshold(31, C = 15)` — 98.8 %
- Маска $\rightarrow$ объект: `inRange` $\rightarrow$ открытие 5×5 $\rightarrow$ закрытие 9×9 $\rightarrow$ `connectedComponentsWithStats` $\rightarrow$ крупнейшая $\rightarrow$ `bbox`; слипшиеся режет `watershed`, в L7 к порогу добавится гистерезис

САМОПРОВЕРКА

- Почему красный объект в OpenCV ищут двумя масками — и какими числами задаются их границы по H?
- Кадр, где все 256 кодов заняты поровну: что сделает с ним `equalizeHist`, а что — `CLAHE` с `clip 2`?
- Объект — 1 % площади кадра на градиентном фоне: какой порог даст Otsu и почему; что взять вместо?
- Маска с крапинками 2 px и дырами 6 px: какие две операции и с какими ядрами; что изменится, если поменять их порядок?

Разбор — в начале L4, на слайде «Опоры»

ДАЛЬШЕ

- **Занятие 3** (вт 15.09): цветной трекер на видео — `inRange` → морфология → компоненты → `bbox`; четыре бинаризации текста; ★ подбор HSV по клику, GrabCut
- **Мост**: заготовка начинается ровно на демо 2.1 и 2.2 этой лекции
- **ДЗ 1 «Фотолаборатория»** — выдача 14.09, дедлайн **W6**: хромакей, CLAHE против `equalizeHist`, гистограммы до/после
- **Quiz 0** (без оценки, L1–L2) — на этой неделе в Moodle; **Quiz 1** (блок I, L1–L3) — на занятии 4

Ноутбук и данные — в репозитории за сутки до пары

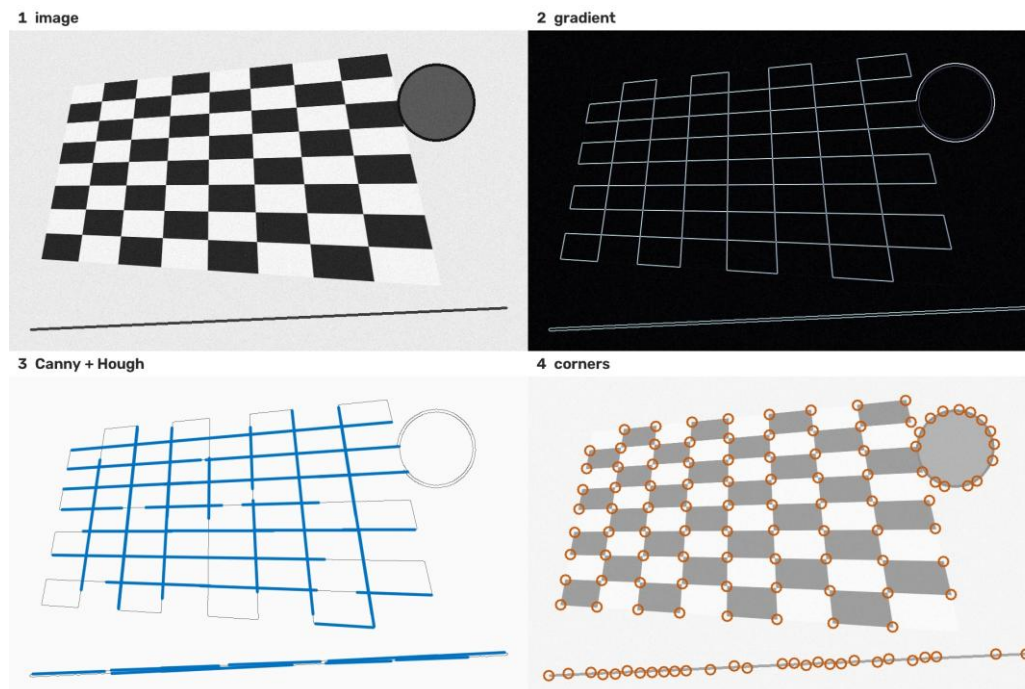
REFERENCES

- Szeliski R. *Computer Vision: Algorithms and Applications*, 2nd ed., 2022 — §2.3.2 цвет, §3.1 точечные операции и эквализация, §3.3.3 бинарная морфология и компоненты, §7.5 сегментация. К следующей паре: §3.6 (геометрические преобразования)
- Гонсалес Р., Вудс Р. *Цифровая обработка изображений*, 3-е изд., Техносфера, 2012 — гл. 3 (гистограммы), 9 (морфология), 10.3 (пороги, Otsu)
- Otsu N. *A Threshold Selection Method from Gray-Level Histograms*, IEEE Trans. SMC, 1979 — четыре страницы первоисточника
- Zuiderveld K. *Contrast Limited Adaptive Histogram Equalization*, Graphics Gems IV, 1994 · Rother C., Kolmogorov V., Blake A. *GrabCut*, SIGGRAPH 2004 · Achanta R. et al. *SLIC Superpixels*, IEEE TPAMI, 2012
- OpenCV docs 4.x: *Color conversions* (cvtColor), *Histograms* (calcHist, equalizeHist, createCLAHE), *Image Thresholding*, *Morphological Transformations*, connectedComponentsWithStats
- Wyman C., Sloan P.-P., Shirley P. *Simple Analytic Approximations to the CIE XYZ Color Matching Functions*, JCGT, 2013 — кривые слайдов 7–8

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
метамеры (metamers)	разные спектры с одинаковым откликом трёх кривых: один RGB, различить нельзя
цветовое пространство (color space)	обратимая замена координат цвета; информации не добавляет, удобство меняет
оттенок (hue)	угол на цветовом круге; в OpenCV 0...179; у серого не определён — шум
насыщенность (saturation)	чистота цвета: 0 у серого, 255 у спектрального; порог по ней отсекает шумный hue
Lab, ΔE	пространство CIE 1976, где расстояние $\Delta E \approx$ различию для глаза; $\Delta E 2.3$ — едва заметно
точечная операция (point operation)	новый код зависит только от старого в том же пикселе: $y = f(x)$, без соседей
таблица подстановки (LUT)	256 готовых значений f — любая точечная операция за один проход
гистограмма (histogram)	число пикселей на каждый код; видит экспозицию, контраст, клиппинг, не видит положение
эквализация (histogram equalization)	точечная операция $T = 255 \cdot \text{cdf}$: коды заняты равномерно, cdf — прямая
CLAHE	эквализация по плиткам с пределом высоты гистограммы: локальный контраст без взрыва шума
метод Оцу (Otsu's method)	порог, максимизирующий межклассовую дисперсию двух горok гистограммы
адаптивный порог (adaptive threshold)	порог = локальное среднее - C: следует за светом, ломается внутри крупных объектов
структурирующий элемент (structuring element)	окно-форма морфологии; эрозия и дилатация решают, «влезло ли» оно в маску
открытие / закрытие (opening / closing)	эрозия $\rightarrow$ дилатация убирает крапинки; дилатация $\rightarrow$ эрозия зашивает дыры; размер прежний
связная компонента (connected component)	пиксели маски, соединённые соседством 4 или 8; по ней — площадь, bbox, центрoид

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: [lectures/L03-color-histograms/demo/](#)

Занятие 3 — принесите цветной предмет для трекара